
Durin's Code

Test Report

CS4031 — Compiler Construction
Spring 2026

Student Name	Student ID
Huzaifa Abdul Rehman	23K-0782
Abdul Moiz Hussain	23K-0553
Muhammad Abdullah Khan	23K-0607

Contents

1	Test Summary	2
2	Lexer Tests	2
3	Parser Tests	3
4	Semantic Analyser Tests	4
5	TAC Generator Tests	5
6	Code Generator Tests	5
7	Virtual Machine Tests	6
8	End-to-End Smoke Test	6
9	Debug Mode Output	7
10	Semantic Error Demo	8

1 Test Summary

All 49 tests pass with zero failures. Total execution time: < 0.2 seconds.

Suite	Binary	Tests	Status
Lexer	lexer_tests	12	PASS
Parser	parser_tests	15	PASS
Semantic	semantic_tests	10	PASS
TAC	tac_tests	4	PASS
Codegen	codegen_tests	3	PASS
VM	vm_tests	5	PASS
Total		49	49 / 49

```
$ cd build && ctest --output-on-failure
Test project .../DurinsCode-main/build
 1/49 LexerTest.TokenizesSampleWorldScript      Passed    0.00s
 2/49 LexerTest.RecognizesKeywordsAndPunctuation Passed    0.00s
 ...
49/49 VMTest.ConditionalSkippedWhenRoomWrong    Passed    0.00s

100% tests passed, 0 tests failed out of 49
Total Test time (real) = 0.13 sec
```

Listing 1: ctest output (abbreviated)

2 Lexer Tests

Test Name	What It Verifies
TokenizesSampleWorldScript	Full world script tokenises correctly end-to-end; checks first 15 tokens including room, string, {, description, item, identifier, number, comma, and colon.
RecognizesKeywordsAndPunctuation	All 14 keywords (room through current_room) plus all punctuation tokens are emitted with the correct TokenType.
RecognizesStringAndNumber	A string literal including spaces and a multi-digit integer are tokenised with the correct type and lexeme text.
RecognizesCurrentRoomCondition	The sequence if current_room == "bag_end" { produces TOKEN_IF, TOKEN_CURRENT_ROOM, TOKEN_EQUAL_EQUAL, TOKEN_STRING, TOKEN_LEFT_BRACE in order.

Test Name	What It Verifies
RecognizesPlayerHasItemAsContextSensitivePlayerAttribute	Identifier <code>has_item</code> is emitted as <code>TOKEN_PLAYER_ATTR</code> (not <code>TOKEN_IDENTIFIER</code>), confirming the context-sensitive dot logic.
RecognizesPlayerAttributeAndPlus	<code>Player.inventory += the_ring</code> produces <code>TOKEN_PLAYER</code> , <code>TOKEN_DOT</code> , <code>TOKEN_PLAYER_ATTR</code> , <code>TOKEN_PLUS_EQUAL</code> , <code>TOKEN_IDENTIFIER</code> .
ReportsUnexpectedCharacter	The character <code>@</code> produces a <code>TOKEN_ERROR</code> whose message contains "Unexpected character".
AcceptsUnderscoreLeadingIdentifiers	<code>_private</code> , <code>_magic</code> , and <code>_123</code> are tokenised as <code>TOKEN_IDENTIFIER</code> . Also validates that column numbers are tracked correctly (<code>_private</code> at column 1, <code>_magic</code> at column 10).
RejectsFloatingPointNumbers	<code>3.14</code> produces <code>TOKEN_ERROR</code> with a message containing "Floating-point".
RecognizesLogicalOrOperator	<code> </code> produces <code>TOKEN_OR_OR</code> .
ReportsColumnNumbers	In the source <code>"room\n @\naction"</code> , the <code>@</code> error token is reported at line 2, column 3.
HandlesWindowsLineEndings	<code>"room\r\n\"test"</code> tokenises <code>room</code> on line 1 and the string on line 2, confirming <code>\r\n</code> is treated as a single newline.

3 Parser Tests

Test Name	What It Verifies
ParsesFullWorldScript	The canonical two-room, two-action world from the proposal parses without error and produces the correct top-level declaration count.
ParsesSingleRoomDeclaration	A room with a description, one item, one exit parses into a <code>RoomDeclNode</code> with the correct field values.
ParsesItemProperties	Item properties of integer, string, and boolean types are parsed into the correct <code>PropValue</code> variants.
ParsesNpcDeclaration	An NPC declaration with <code>health</code> and <code>hostile</code> properties parses correctly.

Test Name	What It Verifies
ParsesExitDeclaration	<code>exit east "buckland"</code> produces direction <code>"east"</code> and target <code>"buckland"</code> in the <code>ExitDeclNode</code> .
ParsesCurrentRoomCondition	<code>if current_room == "bag_end" { ... }</code> produces an <code>IfStmtNode</code> with a <code>ROOM_CHECK</code> condition node.
ParsesPlayerAssignment	<code>player.win = true</code> produces an <code>AssignStmtNode</code> with the correct target, operator, and value.
ParsesPlayerPlusAssign	<code>player.inventory += the_ring</code> produces an <code>AssignStmtNode</code> with <code>op = PLUS_EQUAL</code> .
ParsesPlayerMinusAssign	<code>player.health -= 10</code> produces an <code>AssignStmtNode</code> with <code>op = MINUS_EQUAL</code> and <code>intValue = 10</code> .
ParsesLogicalOrInCondition	A condition joined by <code> </code> produces a <code>ConditionNode</code> with <code>type = OR</code> .
FailsOnMissingRoomBrace	A room body without a closing <code>}</code> causes <code>hadError = true</code> .
ReportsErrorOnFloatLiteral	A floating-point literal in a property value causes <code>hadError = true</code> .
AcceptsUnderscoreIdentifiers	Room and item names starting with underscores parse without error.
EmptyActionBodyIsValid	An action with an empty body action <code>"noop" {}</code> parses without error.
RoomWithNoBodyElementsIsValid	A room with no items, NPCs, or exits parses without error.

4 Semantic Analyser Tests

Test Name	What It Verifies
RegistersRoomsAndItems	A valid program populates <code>SymbolTable.rooms</code> and <code>SymbolTable.items</code> with the correct names.
ErrorOnDuplicateRoom	Two rooms with the same name produce <code>hadError = true</code> and a diagnostic message containing "Duplicate room".
ErrorOnDuplicateAction	Two actions with the same name produce <code>hadError = true</code> .
ErrorOnExitToUndeclaredRoom	An exit whose target string does not match any declared room produces <code>hadError = true</code> .

Test Name	What It Verifies
ErrorOnDuplicateExitDirection	Two exits in the same room sharing a direction produce <code>hadError = true</code> .
ErrorOnRemoveUndeclaredItem	<code>remove phantom</code> where <code>phantom</code> is undeclared produces <code>hadError = true</code> .
ErrorOnHasItemUndeclaredItem	<code>player.has_item(phantom)</code> where <code>phantom</code> is undeclared produces <code>hadError = true</code> .
ErrorOnCurrentRoomUndeclaredTarget	<code>current_room == "ghost_room"</code> where <code>"ghost_room"</code> is not declared produces <code>hadError = true</code> .
CollectsMultipleErrors	A program with three separate semantic errors produces three diagnostics in a single analysis pass (collect-all-errors behaviour).
ValidFullProgramPassesClean	The complete Middle-Earth demo program passes semantic analysis with <code>hadError = false</code> and zero diagnostics.

5 TAC Generator Tests

Test Name	What It Verifies
PrintStatementEmitsPrintOp	An action containing <code>print "Hello"</code> generates exactly one instruction with <code>op = PRINT</code> and <code>strValue = "Hello"</code> .
IfConditionEmitsJumpIfFalse	An action with <code>if current_room == "x" { ... }</code> generates a <code>CHECK_ROOM</code> instruction followed by a <code>JUMP_IF_FALSE</code> .
AssignStatementEmitsSetPlayerAttrOp	<code>player.win = true</code> generates a <code>SET_PLAYER_ATTR</code> instruction with <code>dest = "win"</code> , <code>boolValue = true</code> .
RemoveEmitsRemoveItemOp	<code>remove the_ring</code> generates a <code>REMOVE_ITEM</code> instruction with <code>src1 = "the_ring"</code> .

6 Code Generator Tests

Test Name	What It Verifies
ProducesValidJSON	The output of <code>generateBytecode()</code> is valid JSON (parseable by <code>nlohmann::json::parse</code> without exception).
WorldContainsRooms	The "world" array in the generated JSON contains an entry for each declared room, with correct "id" and "description" fields.
ActionsContainInstructions	The "actions" array contains one entry per action declaration, each with a non-empty "instructions" array.

7 Virtual Machine Tests

Test Name	What It Verifies
LoadsBytecodeFromString	<code>VM::loadBytecodeFromString()</code> with a minimal valid JSON string returns <code>true</code> and does not throw.
ExecutesPrintAction	An action containing a PRINT instruction outputs the expected string to <code>stdout</code> when executed.
ActionNotFoundReturnsFalse	Requesting an action name that does not exist returns <code>false</code> from <code>executeAction()</code> without crashing.
ConditionalActionAddsToInventory	An action guarded by <code>CHECK_ROOM</code> executes its body (and adds an item to inventory via <code>SET_PLAYER_ATTR</code> with <code>+=</code>) when the player is in the correct room.
ConditionalSkippedWhenRoomWrong	The same action does <i>not</i> add the item to inventory when the player is in a different room, confirming that <code>JUMP_IF_FALSE</code> branches correctly.

8 End-to-End Smoke Test

The following session demonstrates a complete compile-and-run using the Middle-Earth demo program (`examples/04_middle_earth.dc`). Commands typed by the player are shown after the `>` prompt.

```
$ ./durinsc ../examples/04_middle_earth.dc

=== Durin's Code Adventure ===
Commands: look, inventory, go <dir>, quit, or any action name.
```

```

=== bag_end ===
The cozy hole of a Hobbit. A golden ring glints on the table.
Items here: the_ring
Exits: east -> buckland

> take ring
The Precious is yours.

> inventory
Carrying: the_ring

> go east
=== buckland ===
A quiet riverside village. The old ferry creaks at the dock.
Exits: west -> bag_end, east -> mount_doom

> go east
=== mount_doom ===
The air is thick with ash. The fiery Cracks of Doom loom ahead.
NPCs here: sauron
Exits: west -> buckland

> destroy ring
The ring is consumed by fire! Middle-earth is saved.

*** You win! The quest is complete. ***

```

Listing 2: Full win-path game session

9 Debug Mode Output

The `--debug` flag dumps the symbol table and optimised TAC before running:

```

$ ./durinsc ../examples/01_hello_world.dc --debug

=== Symbol Table ===
  room:    end
  room:    start
  action:  press button

=== TAC (optimized) ===
action "press button":
  t0 <- [] start
  L0 <- [t0]
    <- [] Hello, adventurer! Your journey begins.
  L1 <- []
  L0 <- []
    <- [] There is no button here.
  L1 <- []

```

Listing 3: Debug mode output for 01_hello_world.dc

10 Semantic Error Demo

Compiling `examples/05_error_demo.dc` produces five semantic errors and zero warnings, confirming the collect-all-errors behaviour:

```
$ ./durinsc ../examples/05_error_demo.dc

[ERROR] line 14: Duplicate room: "dungeon"
[ERROR] line 23: Exit target "sky" not declared
[ERROR] line 30: Duplicate exit direction "north" in room "
    courtyard"
[ERROR] line 35: remove "phantom_item" -- unknown item
[ERROR] line 40: has_item "phantom_item" -- unknown item

Compilation failed: 5 error(s).
```

Listing 4: Semantic error output